check-registry

The check registry and its meta-check

`scripts/check-registry.tsv` + `scripts/verify-check-registry.sh`

Why

`specs/001-workshop-curriculum-platform/spec.md` states two success criteria quantified over *every* automated check:

- **SC-012** — every check has a paired demonstration that it FAILS when the guarded condition is broken. 100%, no exceptions.
- **SC-013** — the system distinguishes "unable to verify" from "passed" and "failed" in 100% of its checks.

Neither is evaluable without an enumeration of "every check". Per §11.4, a PASS that cannot be evidenced is a bluff, so an unevaluable universal claim is worse than no claim. The registry is that enumeration; the meta-check is what keeps it true instead of aspirational.

The concrete motivation is measured, not theoretical. On 2026-09-01 two gates in this tree were found with proofs that could not do their job:

- `scripts/verify-governance-cascade.sh --prove-failure` — its pre-flight control runs against the LIVE tree. The live tree had a real C8 violation, so the control failed, the mutation battery ran **zero** mutations, and the command exited 1 having proved nothing.
- a second gate exercised only sandboxed copies while its real entry point could not start.

Neither is visible from a summary line. `--run-proofs` sees both.

Format

TSV, TAB-separated, field 1 is the row type from a closed vocabulary:

```
scanroot   <dir>
exempt     <path>  <reason>
check      <id>  <entry-point>  <proof-kind>  <proof-arg>  <undet-
probe>
debt       <id>  <entry-point>  <owed>        <reason>
```

Full field semantics, including why TSV rather than YAML, are in the header of `scripts/check-registry.tsv` itself. In short: the governance submodule already keeps every gate ledger as TSV parsed with `cut -f1`, typed rows from a closed vocabulary are already a governance requirement here (`submodules/constitution/scripts/gates/cm_ledger_row_typed_from_closed_vocabulary.sh`), and TSV needs no `python3`/`yq` — a pre-push instrument must not have a parser that can fail to load and then report a tree it never read.

Anti-drift

A hand-maintained list rots. Both directions are closed:

- **outward** — every registered path must exist; a dead row is a FAIL.
- **inward** — every `*.sh` under a declared `scanroot` must appear in some row. A new check dropped into `scripts/` and not registered is a **FAIL**, not an omission. This is the half without which the registry is decorative.
- **ratchet** — a `debt` row whose owed property is found SATISFIED is a FAIL (“stale debt — promote it”). Debt may hide a gap; it must not hide a fix.

Exit codes

`0` all registered checks conform · `1` a real conformance violation · `2` could not determine (registry missing/unreadable/malformed, scanroot absent, probe timed out). The instrument is not exempt from its own SC-013 rule.

Registered **debt** does not block a default run. It prints on every run and is never reported as compliance. `--strict` turns each debt row into a failure.

Modes and cost

mode	what it does	measured wall clock
default	proof STRUCTURE + executes every rc-2 probe	**1.3 s** (2026-09-02)
`--prove-failure`	its own §1.1 paired mutation proof (10 mutations)	**5.6 s**
`--run-proofs`	additionally EXECUTES every registered paired proof	**see below**

The `--run-proofs` figure of **243 s** was measured on 2026-09-01 over 13 check rows. Five more were added on 2026-09-02 (see "The conformance gap" below) and they are not free: `setup-agents-wizard-suite` alone runs its 217-assertion subject six times. Re-measure rather than quoting the old number; it is a pre-tag / release-sweep instrument, not a hook.

The trade is declared rather than silently sampled: executing every paired proof is the strongest evidence and is far too slow for a hook, so the default run verifies structure and says in its own summary that it observed no proof actually run. Nothing is skipped at random.

Wiring it into the pre-push hook

Not wired by this change — that is the controller's call once the debt below is scheduled. When wired, it belongs in `scripts/pre-push-gates.sh` as a new gate id in `GATE_IDS`, with:

```
gate_N() { bash "$ROOT/scripts/verify-check-registry.sh"; }
# name:       check-registry conformance (SC-012 / SC-013)
# provenance: docs/check-registry.md
```

Two cautions specific to that host:

1. ~~`pre-push-gates.sh` maps **every** non-zero child rc to `FAILED`.~~ **WITHDRAWN, re-verified 2026-09-01.** `run_gate` now has a distinct `UNDET` verdict for a child rc=2 (around lines 512-518), the summary prints `undetermined=` as its own counter, and an undetermined run says "COULD NOT DETERMINE a verdict" and blocks *without* accusing the tree. A `verify-check-registry.sh` rc of 2 is therefore reported as what it is. The debt row below was narrowed to `proof` accordingly, and on 2026-09-02 that remaining debt was cleared too. **That withdrawal is no longer read off the source.** It is MEASURED, by `bash scripts/pre-push-gates.sh --prove-failure` mutation M2/M2b: a child gate seeded to return rc=2 makes the runner print `passed=1 failed=0 undetermined=1`, say "NOT a failure of this tree and NOT a pass", and exit 1. Re-introducing the old conflation is caught by 8 of the 9 mutations.
2. Run it **default mode** in the hook (≈1.3 s). `--run-proofs` does not belong on a push path; it belongs in a pre-tag / release sweep.

The conformance gap — CLOSED 2026-09-02

Determined empirically by running each entry point, not by reading its prose. Re-derive with `bash scripts/verify-check-registry.sh`.

The five DEBT rows this section used to carry are cleared. The registry now declares ZERO debt. Measured 2026-09-02:

```
bash scripts/verify-check-registry.sh
# 41 PASS, 0 FAIL, 0 DEBT, 0 UNDET, 0 NOTE  -> exit 0
```

The earlier figure — **29 PASS / 0 FAIL / 5 DEBT / 0 UNDET, exit 0** on 2026-09-01, and **31 PASS / 5 DEBT** once `remedy-executability` was registered — is superseded, not wrong-at-the-time. PASS counts *assertions* (each conforming check contributes an SC-012 row and an SC-013 row), not checks: 18 check rows plus the R0/R1/R1b/R2/R5 structural rows.

The five that were owed, and what each now ships. Every one was verified in **both** directions — it passes on this tree, **and** it fails when the gate it guards is deliberately weakened, which is the only evidence that separates a proof from a decoration:

check	paired proof	measured
	`--prove-failure` — synthetic consumer	10 mutations + 1 stated LIMIT, **7 s**; a

check	paired proof	measured
`verify-all-constitution-rules.sh`	tree whose child gates read their exit code from a sidecar file	sweep that no longer fails on a failing or blind child gate is caught by **6** of them
`lumen-index-doctor.sh`	`--prove-failure` — a throwaway sqlite index built with plain sqlite3 (no sqlite-vec)	10 mutations, **42 s**; raising the duplicate threshold out of reach and disabling the per-vector tests is caught by **5**
`ollama-tune.sh`	`--prove-failure` — a PATH-shimmed synthetic host (`systemctl`, `journalctl`, `curl`, `podman`, `docker`, `pgrep`, `ss`, `sudo`, `systemd-path`)	10 mutations, **13 s**; collapsing could-not-determine into “fine” is caught by **7**
`pre-push-gates.sh`	`--prove-failure` — a throwaway git repository with two nested submodules, driven through `PREPUSH_ONLY`	9 mutations, **4 s**; re-introducing the rc-2→FAILED conflation and removing the push block is caught by **8**
`test-setup-agents-wizard.sh`	`--prove-failure` — the real wizard, byte-copied, mutated four ways; plus its own rc-2 states	8 mutations, **3 m 32 s**; making a FAILED assertion stop reddening the suite is caught by **8**

`test-setup-agents-wizard.sh` also owed a **three-valued exit of its own**, and now has one: `--root <dir>` that is not a directory, a wizard that is absent, unreadable or empty, and an evidence directory that cannot be created are each **rc 2**. It previously asserted three-valued behaviour in the wizard it tests (K23b/K25) while implementing none for itself.

The gap the promotion did not close — CLOSED 2026-09-02

The old `constitution-rules-sweep` debt row named a specific missing demonstration: *"nothing demonstrates the SWEEP itself fails when a child gate is silently dropped."* It was split into two halves, and **both are now closed**:

- **ADD direction — closed.** Mutation M9 drops a new gate file into the gates directory and measures the discovered count moving `3 -> 4`, with the added gate's failure reddening the sweep and **no edit to the sweep**. §11.4.32's "gates are DISCOVERED, never hardcoded" is therefore measured, not asserted.
- **DROP direction — closed.** The assertion that used to be called **L1** recorded the OPPOSITE result: it deleted a gate file, measured the count falling `3 -> 2`, and recorded that the sweep **still exited 0**, because there was no expected-gate ledger to compare against. It is now **M11**, and it requires the sweep to exit **1** and to NAME the vanished gate. Measured 2026-09-02: *"the count fell 3 -> 2, the sweep exited 1 and NAMED the vanished gate."*

The ledger, and why it is not a hardcoded list

The blocking design problem was stated in this section and has not gone away: **the gate population moved 57 → 286 when the constitution was fast-forwarded**, so any expected list written into the sweep is wrong at the next pin — and a gate that is wrong at every bump gets deleted or made advisory the first time it fires. The expected set is therefore DATA, not code:

`scripts/constitution-gate-ledger.tsv` — TAB-separated, typed rows from a closed vocabulary (`pin`, `gate`), the same shape `check-registry.tsv` and the constitution's own gate ledgers use. It records the constitution HEAD the baseline was taken at plus one row per discovered gate (286 at pin `3be10826f3d2`, measured 2026-09-02). It is regenerated only by an explicit `bash scripts/verify-all-constitution-rules.sh --update-ledger`, which prints what changed — so a population change always lands in a reviewable git diff.

The discriminator — "the pin moved" vs "a gate vanished" — uses two instruments, and the second is what makes the separation real:

1. **The pin.** The population is a function of the constitution's commit. If the ledger's pin EQUALS the live pin, a ledgered gate that is

missing **vanished**. Determined, no git query needed, and this is the case that holds on a tree nobody has bumped.
2. **Git, when the pin HAS moved.** A missing path that git still reports as TRACKED at the current commit was deleted from the working tree — a vanished gate, **determined**, however far the pin travelled. A missing path that is NOT tracked at the current commit is a gate upstream no longer carries — a legitimate population change, reported as a NOTE and not gating.

Both directions are proved, because either one alone would be worthless: **M13** requires an upstream removal across a moved pin NOT to gate (else the ledger cries wolf on every bump and gets deleted), and **M14** requires a local deletion across a moved pin to STILL be caught and named (else the ledger is a rubber stamp that any bump switches off). **M12** requires a deleted ledger, and **M12b** a malformed one, to be reported as could-not-determine — deleting the ledger must not buy back the old silence. **M15** requires `--update-ledger` to restore a clean state deliberately.

The residual limit, stated rather than asserted away (§11.4.6): when the pin has MOVED *and* the constitution is not a git work tree (a vendored or archive-extracted copy), neither instrument applies and attribution is genuinely undecidable. The sweep reports that as could-not-determine and exits non-zero — a blind instrument is never a pass (§11.4.201(7)(b)) — rather than guessing in either direction. M13/M14 require git and SKIP with a reason when it is absent. A second, smaller limit: a gate deleted upstream and re-added under a different name in the same bump reads as one removal plus one addition. The ledger does not track renames, and does not claim to.

Scope boundary: the ledger covers the gates discovered under `submodules/constitution/scripts/gates/**`. The project-side gate run in step 2b (`tests/test_constitution_inheritance.sh`) is a fixed known path already handled by an explicit §11.4.3 skip-with-reason, and is not ledgered.

The inoperative-proof defect — closed for two, found in a third (2026-09-01)

`--run-proofs` used to report `governance-cascade` and `manifest-pins` as proofs that could not pass. Both ran their control against the **live tree**, so any real violation made the control fail and the battery never started. Measured by copying each script into a tree that was not a green checkout: both exited non-zero having run **0 mutations**.

Both are now **FIXED**. Each has a synthetic control that is green by construction, and the live run is demoted to a REPORTED pre-flight that cannot disable the battery — the shape `verify-check-registry.sh --prove-failure` uses. Re-measured against the same red tree: `manifest-pins` runs **10/10** mutations, `governance-cascade` runs **11/11**. The same shape was used for the two new audit proofs (10 mutations each).

A **third** instance is now visible and is *not* fixed: `continuation-sync` (`scripts/continuation-check.sh --prove-failure`) also baselines against the live tree, so it reports `PROOF FAIL baseline, unmutated` whenever `CONTINUATION.md` is stale — which it is today, because commits `284adfa3`, `b2397886` and `6fd4cb2a` changed watched governance files without updating it in the same commit (§12.10 protection 2). (Those three have been renamed twice by authorized content-boundary rewrites: `d0b3c64`/`96b2988`/`ee3933d` → `7b4df26d`/`4ee9e8de`/`b0ab4b44` on 2026-09-01 → the values above on 2026-09-02. Only the last generation resolves; the mappings are in `docs/content-boundary-incident-2026-09-01.md` §8B and §11.4.) It is a *degraded* rather than an inoperative proof: its six mutations and its rc-2 branch all still execute and pass. It was deliberately left alone because a synthetic control for it must make C7 (the §6 gate table vs the live runner) and C8 (production workflow facts) green by construction, and the runner it compares against — `scripts/pre-push-gates.sh` — was under concurrent edit.

Consequence, stated rather than implied: `bash scripts/verify-check-registry.sh` is **rc 0**, but `--run-proofs` is **rc 1** on account of that one row.

The 2026-09-02 `--run-proofs` split — rc 1, and NOT for that reason any more

Re-measured 2026-09-02 immediately after the five debt rows were cleared:

```
bash scripts/verify-check-registry.sh --run-proofs
# 54 PASS, 5 FAIL, 0 DEBT, 0 UNDET, 0 NOTE  -> exit 1
# measured twice the same day: 10 m 44 s, then 10 m 17 s — same split both times
```

All five newly-promoted checks PASS under `--run-proofs` — each proof was executed and each reported real mutation results. `continuation-sync` now passes too. The five FAILs are a *different* set, and every one of them belongs to a check this change did not touch: `git diff` reports `scripts/verify-check-registry.sh`, `verify-provider-ci.sh`, `verify-submodule-remote-sync.sh`, `verify-mutation-anchors.sh`,

`verify-private-object-exposure.sh` and `verify-remedy-executability.sh` all **byte-identical to HEAD**, and the registry diff adds and removes only the five rows named above. They are recorded here rather than fixed, because fixing them means editing gates that belong to other work in flight:

row	what `--run-proofs` says	measured cause
`provider-ci`	`--selftest` exited rc=1	a real assertion failure inside that selftest: `M6b no repository or owner name in body — expected 0 names present, got 1 present`. A genuine finding about that gate, not about this one.
`submodule-remote-sync`	"exited 0 but reported no mutation results"	**a false positive of the meta-check's own heuristic.** The proof runs and returns 0; the meta-check appends `--quiet` because that script *has* a `--quiet` case arm, and `--quiet` suppresses the per-case lines the heuristic looks for. Measured: `--prove-failure --quiet` → rc 0, 23 lines, 0 needle matches.
`mutation-anchor-rot`	same	**same class, different trigger.** These three have no `--quiet` arm, so they run bare — and their summaries are prose (*"rot was DETECTED (1)"*, *"an absent*

row	what `--run-proofs` says	measured cause
		remedy was CAUGHT (1)") with neither `M<n>` labels nor the phrase "*N* mutations". Measured bare: rc 0, 0 needle matches.
`private-object-exposure`	same	as above
`remedy-executability`	same	as above

The heuristic is `(^|[^A-Za-z])M[0-9]` or `[0-9]+\s+(mutation|mutations|drift| drifts|caught|passed)`. Four working proofs simply do not speak that way. **The right fix is to make those four proofs say what they did — not to loosen the heuristic**, which is the only thing standing between this registry and a proof that returns 0 without exercising anything. That is left to the owners of those gates, with the measurement above so nobody has to re-derive it.

Declared scope boundary

The scanroots are `scripts/` and `tests/`. `_tools/` and `_tests/` hold further gates that `pre-push-gates.sh` runs (`_tools/audit-hardcoding.sh`, `_tools/translate/reproducibility-selftest.sh`, `_tools/portfolio/self-validate.sh`, `_tests/run-harness-selfvalidation.sh`) and are **not** swept yet. That is a declared boundary, not a claim that they conform. Widening it is a one-line `scanroot` addition.

The submodules are a HARDER boundary, and it is not one line

Widening a scanroot into a submodule is **not** the same one-line change, and the difference was measured rather than assumed on 2026-09-03. R0 makes an absent scanroot **rc 2**, and `workshop/` is a **private** submodule, so on any clone that cannot initialise it the directory is empty. Measured in a sandbox copy of this tree with `scanroot workshop/pipeline` appended and an empty `workshop/`:

```
bash scripts/verify-check-registry.sh --root <sandbox>
◍ UNDET [REGISTRY] declared scanroot 'workshop/pipeline' is not a
```

```
directory
  under <sandbox> — the sweep cannot be performed
COULD NOT DETERMINE — the registry could not be read, so nothing
was verified.
rc = 2
```

One added row takes the **entire** umbrella meta-check dark — not just the added rows — for every reader without private access. `REGISTRY` is also hardcoded to `$ROOT/scripts/check-registry.tsv`, so this instrument cannot be pointed at a second file without editing it.

Feature-001's **transcription pipeline** gates are therefore enumerated by their own registry **inside** the private submodule, referenced here by path only:

artefact	path (inside the `workshop` submodule)
registry	`platform/gates/check-registry-001.tsv`
meta-check	`platform/gates/verify-check-registry-001.sh`
paired proof	`platform/gates/prove-check-registry-001.sh`

It carries the same closed-vocabulary TSV shape and the same three-valued exit, uses this file's hollow-proof heuristic **unchanged**, and its R5 sweep is **recursive** with declared `prune` rows — stricter than the `-maxdepth 1` sweep here, because two of the gates it registers live one directory down. It is run automatically by that module's `scripts/verify.sh`, which discovers every `platform/gates/verify-*.sh` at run time.

`submodule-remote-sync` — added 2026-09-01

field	value
row	`check submodule-remote-sync scripts/verify-submodule-remote-sync.sh flag --prove-failure --root /nonexistent`
entry point	`bash scripts/verify-submodule-remote-sync.sh`
paired proof	`bash scripts/verify-submodule-remote-sync.sh --prove-failure`
rc-2 probe	`--root /nonexistent`

What it closes

It closes a blind spot that was structural, not accidental. Cascade check C9 and `scripts/verify-manifest-pins.sh` both answer *"does `helix-deps.yaml` agree with the gitlink this repository will commit?"* — a purely **local** comparison. Neither has ever contacted a remote: on 2026-09-01 a grep for `ls-remote` and `git fetch` across `scripts/*.sh` returned **nothing**. So both exited **0** all day while `submodules/constitution` sat behind its upstream, and the root carriers had to warn readers in prose that *"the pin is BEHIND its upstream, and no gate in this tree catches that"*. Prose is not a gate.

The two instruments answer different questions and neither replaces the other:

instrument	compares	question
`manifest-pins` (C9)	manifest `ref` vs local gitlink	is the RECORD true?
`submodule-remote-sync`	local gitlink vs remote tip	is the PIN fresh?

Both can be green while the tree is stale. Only together do they mean "what we recorded is what we will commit, and what we will commit is what upstream has".

Contract

Three-valued, and **state 2 outranks state 1**:

- **0** — every owned gitlink was compared and equals its remote tip.
- **1** — at least one owned gitlink differs, reported as **BEHIND** (fast-forwardable), **AHEAD** (the gitlink is an UNPUSHED commit, so the tree does not clone for anyone else) or **DIVERGED** (operator reconciliation; never automate). These have different remedies and are never conflated.
- **2** — COULD NOT DETERMINE: offline, DNS failure, SSH auth refused, remote gone, probe timeout, or `git` absent. **A network failure must never read as "current"** — that would replace an old blind instrument with a new one.

The fleet is **derived** on every run from `.gitmodules` plus `helix-deps.yaml`; there is no roster in the file. Ownership is classified from evidence — a path documented in the manifest's third-party comment block is reported but never gates, because upstream's release cadence

is not ours to fix. Measured 2026-09-01 that classifies `submodules/superspec` third-party and the other 12 owned, without the script naming either.

First real run, 2026-09-01

exit 1 — 10 CURRENT, 2 DRIFT, 0 UNDETERMINED of 12 owned gitlinks probed, 13 declared. `design-toolkit` and `ai_interviewing` are both **BEHIND** their remotes and fast-forwardable. Neither was visible to any pre-existing gate.

Paired proof

`--prove-failure` builds real local git repositories in a `mktemp -d` sandbox wired by `file://` remotes, so `ls-remote` exercises the real code path with **no network involved**. Six states, each asserted separately: CURRENT→0, BEHIND→1, DIVERGED→1, AHEAD→1, UNREACHABLE→**2 (never 0)**, third-party-behind→0, plus a restored control. The unreachable case is the one an unproven instrument would get wrong, and it is the reason the gate exists.

Additionally seeded against the **live** tree: forcing every remote unreachable (`GIT_SSH_COMMAND=false`) yielded **12 UNDETERMINED, rc=2**, and removing the mutation restored the pre-mutation verdict of rc=1.

`private-object-exposure` — added 2026-09-01

field	value
row	`check private-object-exposure scripts/verify-private-object-exposure.sh flag --prove-failure --root /nonexistent`
entry point	`bash scripts/verify-private-object-exposure.sh`
paired proof	`bash scripts/verify-private-object-exposure.sh --prove-failure`
rc-2 probe	`--root /nonexistent`

What it closes

A hazard that was found, written down, and then guarded by nothing but the sentence that recorded it. `docs/content-boundary-incident-2026-09-01.md` §8A.6 states it exactly:

> *"It is recorded here because a local-any hazard that nothing checks is one accidental `git push --all` away from being a much larger incident than this one."*

(The document's own wording is "a local-only hazard"; the point is unchanged.)

The concrete state, measured 2026-09-01 before the operator acted: the local branch `backup/pre-untrack-git-backup-a8137db` carried **96 files** that were a copy of the PRIVATE `milos85vasic/workshop_curriculum` object store, including a **1,870,772,012-byte** packfile, inside the **PUBLIC** `milos85vasic/vasic`. It was not an ancestor of `main` and `git ls-remote` showed it was never pushed. `.git/hooks/` is untracked, so a fresh clone runs no hook at all. Nothing in the tree could say any of that.

The branch has since been removed (`docs/branch-removal-2026-09-01.txt`), and that changes nothing about this check. It guards the whole **category** rather than that one branch: the §9.3 habit that produced it — copy a submodule's `.git` into the umbrella before a destructive operation — is the correct habit and will produce another one. Not one line of the detection reads a branch name, so the removal did not alter a single code path. What changed is only which state the tree is in, and the check reports that state precisely rather than going quiet.

Why a rename cannot defeat it

The detector reads **no** branch name, directory name, commit message or date. It keys on the internal layout and byte magic that a copied repository must keep in order to still be a repository:

id	evidence	read from
E1	`PACK` + pack version 2/3	blob content
E2	`\377tOc` + index version 2	blob content
E3	zlib stream that inflates to a git loose-object header	blob content

id	evidence	read from
E4	`HEAD` holding `ref: refs/…` or a bare 40-hex sha	blob content
E5	`config` holding `[core]` and `repositoryformatversion`	blob content
E6	`packed-refs` beginning `# pack-refs with:`	blob content

A directory is judged a foreign object store on **≥2 evidence classes, at least one of them an objects-layer proof (E1/E2/E3)**. Path shape is a cheap pre-filter and never a verdict.

Everything found this way is foreign *by construction*: git will not track its own `.git`, so an object store appearing in `git rev-list --objects` output is necessarily a copy of some other repository, committed as content. Provenance, when available, comes from E5 — the captured `config`'s `url =` lines — and is matched against the private half of the fleet.

Scope — the general class, three layers

layer	question	source of truth
A	does any object reachable from **any** ref, or staged in the index, belong to a foreign object store?	`git rev-list --objects --all --indexed-objects` — exactly what `--all` / `--mirror` publish, plus what `git add .` would commit next
B	is a submodule's tree committed as **files** instead of a gitlink, or is its path not a repository checkout at all?	`.gitmodules` paths+urls, corroborated against `helix-deps.yaml`; visibility from `gh`
C	is a foreign object store in the worktree held off by nothing but `.gitignore`?	`find` + the same byte magic, then `git ls-files` / `git check-ignore`

No roster is hardcoded. Measured 2026-09-01 the fleet derives as **13 declared submodules: 10 public / 3 private / 0 visibility-undetermined** — re-derive, do not quote it.

Pushable vs reflog-only, graded apart. Layer A takes *two* enumerations: the pushable set (`--all --indexed-objects`) and the union that adds `--reflog`. `git branch -D` unhooks a store from every ref but leaves its objects in the object database until `gc --prune=now`. `git push` cannot publish an object no ref reaches, so calling that an exposure would be a false finding — and going silent about it would be worse, because one reflog checkout puts it back on a ref. It is therefore an **unsuppressible NOTE at rc 0**, and the paired proof asserts both halves: rc 0, *and* the store still named in the output.

Reading a captured pack without a repository. For each store the check runs `git show-index` on the captured `.idx` blob. That parses a pack index with **no repository and no `.pack` file**, which matters because a captured store's `config` can carry a stale `worktree =` line that makes every ordinary git command abort. On 2026-09-01 three consecutive attempts to read exactly such a store in this tree reported *"0 commits, nothing here"* — and every one of them was an **aborted query**, not a measurement. The census also compares the store's own refs (`packed-refs`, `refs/**`, a detached `HEAD`) against the objects it actually holds, so an **incomplete capture** — a backup that would not restore — is reported as one. A census that cannot be taken prints as *unknown*, never as *empty*.

Contract

Three-valued, and **a confirmed failure outranks an undetermined**:

- **0** — no foreign object store anywhere, no submodule content committed as files, every declared submodule path a real repository checkout.
- **1** — a determined finding.
- **2** — could not determine: no root; root not a repository **toplevel**; `git rev-list` failed; `.gitmodules` present but unparseable; a submodule path that cannot be classified; or a submodule whose state needs grading while its provider visibility is unknown.

When an unknown visibility is deliberately *not* an rc-2, stated so it is never read as leniency: a submodule contributing zero objects to this repository's history whose path is a real checkout has no content here

to expose. No provider answer could change that structural fact, so none is demanded. Visibility is required — and its absence escalated — exactly where it would change the verdict.

The `rev-parse` walk-up trap. `git -C <dir> rev-parse --git-dir` *succeeds* inside an empty directory nested in a repository, because it walks **up** and finds the parent's `.git`; a check written that way reads an uninitialised submodule as healthy. Every repository test here instead requires the directory to **be** the resolved toplevel (`rev-parse --show-toplevel`, both sides made physical with `pwd -P`, compared for equality). The paired proof asserts this directly rather than describing it.

The guard is **read-only**. It never writes, deletes, renames, force-updates or pushes a ref. Removing a backup is an operator decision under §9.3.

Two real runs, 2026-09-01, either side of the operator's removal

Before — exit 1, 2 findings, 0 undetermined:

1. foreign git object store at `workshop-git-backup-2026-08-29`, **6 of 6** evidence classes, largest packfile 1,870,772,012 bytes, provenance `milos85vasic/workshop_curriculum`, carried by `refs/heads/backup/pre-untrack-git-backup-a8137db`;
2. that store is a copy of a **PRIVATE** repository held inside a **PUBLIC** one.

After the branch was removed — exit 0, 0 findings, 0 undetermined, **3 unsuppressible notes**. The same store is still detected, now classified `REFLOG-ONLY`: unreachable from every ref and from the index, therefore not publishable by any `git push`, with its objects still in the object database until a `gc --prune=now`. The gate says so on every run rather than falling silent.

Its census, read from the `.idx` files alone: **8 objects across 2 index files** — the 1.78 GB pack indexes exactly **one**. That is the measured basis for "this was never a history backup".

One correction, recorded because it was arrived at by a narrower read: the store's `packed-refs` names `59460387d331…`, which the **large** pack does not contain — but the **small** pack does (`git show-index` on `pack-`

`ef685738….idx`, line 5). Taken across the whole store, all **2** refs resolve, and the gate reports exactly that. “The pack does not contain it” was true; “the backup would not restore” did not follow from it.

Plus the standing worktree NOTE: a 3,742,068,523-byte copy of the same store sits in the working tree, untracked **only** because of `.gitignore:90 workshop-git-backup-*/`. Given that this project’s commit wrapper runs `git add .` (INVENTORY gap G5), that line is the whole of the protection.

Paired proof

`--prove-failure` builds a throwaway repository under `mktemp -d` with an isolated `HOME` and `GH_CONFIG_DIR`, and never touches the real tree. Measured 2026-09-01, **all assertions pass, rc 0**:

step	assertion	measured
GREEN	ordinary repository	rc 0
RED	a **real** foreign store (verified `PACK` magic `5041434b00000002`) committed at `assets/vendor-data` on branch `wip/notes`	rc **1**
DECOY	`docs/samples` carrying packfile-, idx- and loose-object-**shaped** paths plus `HEAD`, `config` and `packed-refs`, all with non-git bytes	**not** flagged
REFLOG	branch deleted, reflog kept	rc **0**, *and* `REFLOG-ONLY` still named — asserted both ways, because a silent 0 is indistinguishable from a clean tree
GREEN	`reflog expire --expire=now --all` + `gc --prune=now`	rc 0, subject sha256 unchanged, 0 files dirtied
UNDET	`--root /nonexistent`	rc 2
UNDET		rc 2

step	assertion	measured
UNDET	a directory that is not a repository an **empty** submodule dir nested in a repo — `rev-parse --git-dir` returns the parent's `.git` with rc 0	rc 2, and the walk-up is the *named* reason

The `reflog expire` / `gc --prune=now` above run **inside the `mktemp -d` sandbox only**; the guard itself never runs either, and the real tree is not touched by any mode of this file.

Three of those rows are the ones that matter. The **decoy** is what separates content detection from a name match: without it, "a rename cannot defeat this" would be a claim rather than a demonstration. The **empty nested directory** row asserts both the rc and that the uninitialised submodule is the stated cause — an rc 2 arrived at for an unrelated reason would prove nothing about the trap. The **reflog** row is asserted both ways for the same reason in the other direction: an rc 0 that says nothing cannot be told apart from a clean repository.

The mutant's names are asserted at runtime to contain none of `backup`, `workshop`, `git-backup`, `.git`, `objects-copy` or `mirror`; if any did, the proof aborts with rc 2 rather than crediting itself.

Wiring, and what was deliberately not done

This is a **gate**, and the gate is the guarantee. It was **not** wired into `scripts/pre-push-gates.sh`, for two reasons stated rather than implied:

1. `git push --no-verify` bypasses hooks entirely, and `.git/hooks/` is untracked, so a hook is a convenience and never the guarantee. A guard whose only home is a bypassable hook is not a guard.
2. `scripts/pre-push-gates.sh` was under concurrent edit by its owner (the same reason the `prepush-gates` debt row above records for not touching it), and its `GATE_IDS` array is a hardcoded list, so adding a gate means editing that file. Doing so from a change that does not own it is how two agents produce one broken runner.

The wiring, when its owner takes it, is one id in `GATE_IDS` and one `run_gate` arm; the entry point takes no arguments and already returns the 0/1/2 the runner expects. It deliberately does **not** accept the hook's

stdin refspecs: it always scans every ref, because restricting it to the refs being pushed would make it blind to precisely the `--all` / `--mirror` case it exists for.

`remedy-executability` — added 2026-09-01

`scripts/verify-remedy-executability.sh` — **every documented way out must actually exist.**

What it closes

`GET /api/chapters/{chapter}/accuracy` served a correct, honest body:

```
{"measured": false,
 "reason": "verify-accuracy has not been run for this chapter",
 "remedy": "bash workshop/scripts/verify-accuracy.sh <chapter> --
        reference <path>",
 "wer": null}
```

and `ls workshop/scripts/verify-accuracy.sh` reported **no such file**. The endpoint was not wrong about the measurement — it was right that none had been taken. It was wrong about the **way out**, which is the part a reader acts on. Accuracy was not un-measured; it was **un-runnable by the documented command**, and under §11.4.6 that is a bluff: the body asserts a path to resolution that cannot be walked, and the reader cannot tell without trying it.

Nothing in this tree was checking. Every gate here verifies its subject; not one verified its own advice. A remedy string is the only output a gate produces that the reader is expected to **execute**, and it was the only output nothing graded. That is the class this closes — not the single string.

Derivation, and the declared scope boundary

Candidates are derived, never listed. The tree is scanned for the marker forms it actually emits and whatever is found is graded:

Form	Example
structured key	`"remedy": "…"` in JSON and Go map literals
Go const	`const AccuracyRemedy = "…"` (any `*Remedy` identifier)

Form	Example
line marker	line-leading `Remedy:` / `Fix:` / `Run:` / `Try:` / `try:`

A payload-free marker yields no candidate, which is why Python's `try:` — always payload-free — does not flood the corpus.

Scope is declared, following the `scanroot` convention this registry already established: `scripts`, `tests`, `workshop/scripts`, `workshop/platform`, `workshop/pipeline`, `workshop/docs`, `_tools`, `_tests`, `docs`, printed on every run and overridable with `--scanroot`. `ai_interviewing/` and the two site submodules are **out of scope**: their `Fix:` lines are prose about software engineering in general, not remedies this platform emits. A declared boundary is not a claim that what lies outside it conforms. Vendored and generated trees (`node_modules`, `venv`, `vendor`, `pipeline/engines`, `pipeline/models`, `dist`, `_site`, `testdata`, `.specify`) are pruned: counting an upstream's broken advice as this tree's finding would be a claim with no remedy attached, which is this gate committing its own defect.

The gate **does not exclude itself**. Its own header quotes the accuracy endpoint's remedy, and that string is graded on every run like any other.

Command vs instruction — the discriminator, and why it is safe

Not every remedy is a command. *"Ask your administrator to install podman"* is a legitimate remedy and there is nothing to resolve. The rule is stated, not implied:

> A remedy is a **COMMAND** when it names a **subject a shell can start** — a path-shaped token (`dir/file.ext`, `./file`, an absolute path), or a head token that resolves on `PATH` **to an executable**.
>
> A remedy is an **INSTRUCTION** when it names no such subject. It addresses a person; there is nothing here to resolve; it is not a defect.

The property that makes this safe: classification never consults whether the subject exists. Path-shape is lexical, so `bash scripts/nope.sh` is a COMMAND that FAILS — never an instruction that passes.

A broken remedy cannot be reclassified into the harmless bucket by virtue of being broken. That inversion is excluded by construction rather than by care.

Tokens holding a placeholder or an expansion — `<path>`, `${VAR}`, `$1`, `*` — are **not** subjects. They are holes the reader fills, and resolving them would be resolving the gate's own guess.

One narrow prose test refines the ambiguous branch only. The measured case:

```
"remedy": "install a working ffprobe or ffmpeg on the host that
        runs " + …
```

whose head token `install` is coreutils, so a head-resolves rule alone calls an operator instruction a command. A payload carrying bare English articles and prepositions as standalone words is treated as prose. It is applied **only** where no path-shaped subject was found, so it can never hide a broken file remedy.

Existence is not enough — the operation is exercised

Measured on the development host, 2026-09-01:

```
command -v ffprobe   ->  /usr/bin/ffprobe   (found)
ffprobe -version     ->  exit 0             (runs)
ffprobe -show_format ->  exit 8             (UNUSABLE)
```

Present, executable, and unable to do the thing it is named for. So for every **file subject** the gate does not stop at `-e`: readable, non-empty, a regular file; either the executable bit **or** an explicit interpreter named in the remedy itself; that interpreter (or the shebang's) resolved on `PATH`; and then the file is **parsed with that interpreter's own syntax check** — `bash -n` for shell, `ast.parse` for Python. That exercises exactly what the remedy promises, "this can be started", and it catches the present-but-broken case without executing anything.

Relative subjects are resolved from the emitting file **outward to the root**, because a remedy written inside `workshop/` may spell a path from the module root while the same string served over HTTP spells it from the repository root. Both are correct; a gate that knew only one would invent findings.

Honest boundary (§11.4.6). For an EXTERNAL head token — `git`, `podman`, `npm` — the gate proves **existence only**. It does not run them: the corpus contains `git push`, and this repository contains a deploy script that fired at two live production sites when it was run by accident

today. Executing discovered strings is not a safety trade this gate is willing to make. Those are counted, listed under an unsuppressible NOTE, and `--strict` turns them into rc 2. **A NOTE is not a pass; it is a stated limit.**

Contract

Three-valued, and **a confirmed failure outranks an undetermined**:

- **0** — every remedy that names something resolves to something startable.
- **1** — a remedy names a file that is absent, unreadable, empty, not executable with no interpreter given, whose interpreter is missing, or which does not parse. A reader following it would fail.
- **2** — the question could not be answered: `--root` is not a directory, a declared scanroot does not exist, `grep` is absent, or **the scan matched nothing at all**. A gate that inspected nothing has not reported a clean tree; it has failed to run.
- `--strict` additionally promotes the EXTERNAL note to **2**.

First real run, 2026-09-01

exit 0 — 23 remedy strings: **10 resolve, 0 BROKEN, 1 external, 12 instruction**. The ten that resolve include the accuracy endpoint's own `AccuracyRemedy` const, three `ingest.sh` remedies in `chapters.go`, two `extract-videos.sh` remedies in `recording.go`, and `workshop/docs/faq.md`'s `scripts/build.sh`. `--strict` yields **2** on the one external remedy (`git`), as designed.

Two defects in the gate itself were found and fixed by that first run, and both are recorded here because a gate that invents subjects grades nothing:

1. **Glob expansion.** Payload tokenisation uses an unquoted expansion, so a payload containing `**` (markdown emphasis — every `**Run:**` line in these docs) globbed against the working directory. The gate reported a remedy naming `AGENTS.md`, a file that appears in no remedy anywhere. Fixed with `set -f`.
2. **`command -v` is not "is a command".** `command -v AGENTS.md` succeeds on this host, because a directory holding a markdown file is on the operator's `PATH`. Resolution now requires `type -t` plus an executable, non-directory target.

Paired proof (§1.1)

`--prove-failure` builds a throwaway lab — never the real tree — and asserts six states, each separately:

State	Expected
remedy names a script that exists and starts	0
remedy repointed at a nonexistent script	**1**, not 2
restored byte-identically (checksum-proved)	0
second, independent mutation: subject present but unparsable	**1**
operator instruction (`ask your administrator to install podman`)	0, and classified INSTRUCTION
empty scan	**2**

The mutant is asserted to still **parse** (`bash -n`) before the gate is run against it — a gate that only reddens on a syntax break proves nothing. The second mutation exists because absence and unstartability are different failures, and a gate catching only the first would pass a remedy naming a script that exists and is broken.

The strongest evidence is not synthetic. The gate was run against a reconstruction of this repository's own pre-fix state — `accuracy.go` beside every `workshop/scripts/*.sh` **except** `verify-accuracy.sh`:

```
MISSING  workshop/platform/backend/internal/api/accuracy.go:26
         | remedy:  bash workshop/scripts/verify-accuracy.sh
<chapter> --reference <path>
         | subject: workshop/scripts/verify-accuracy.sh — resolves
nowhere
rc=1
```

Adding the script back to the same lab and re-running the same command yields **rc=0**. The mutation is the real history, not an invention.

Wiring, and what was deliberately not done

Registered as a `check` row; **not** wired into `scripts/pre-push-gates.sh`, for the same two reasons the `private-object-exposure` section records: a hook is bypassable by `git push --no-verify` and `.git/hooks/` is untracked, so the hook is never the guarantee; and `pre-push-gates.sh` carries a hardcoded `GATE_IDS` array and was under concurrent edit by its owner. The wiring, when its owner takes it, is one id and one `run_gate` arm — the entry point takes no arguments and already returns the 0/1/2 the runner expects.

`content-boundary` — `--include-untracked` added 2026-09-02

The registry row is UNCHANGED, and that is a statement, not an omission.

```
check   content-boundary    scripts/verify-content-boundary.sh
flag    --prove-failure --root /nonexistent
```

Entry point, proof kind, proof argument and rc-2 probe are all exactly what they were: the check grew a MODE, not a contract. The row is re-verified rather than assumed — measured 2026-09-02, `bash scripts/verify-check-registry.sh` exits **0** at **41 PASS, 0 FAIL, 0 DEBT, 0 UNDET, 0 NOTE** in 2 s, with

```
   PASS  [content-boundary] SC-012 paired proof: '--prove-failure' is a real case arm in scripts/verify-content-boundary.sh and is non-trivial
   PASS  [content-boundary] SC-013 three-valued exit: probe '--root /nonexistent' -> rc 2, distinct from 0 and 1
   PASS  [R5] anti-drift: every *.sh under scripts tests is registered as a check, debt, or exemption
```

R5 stays green because no new `*.sh` was added — the mode lives inside the already-registered script. That run verifies proof STRUCTURE only and says so; the execution evidence is below.

The hole this closes

Every candidate file in that gate was enumerated with `git ls-files`, which lists **tracked files only**. An untracked file in a public working tree was therefore not filtered out — it was **never opened**.

That is not a hypothetical. On 2026-09-02 an untracked file in this public umbrella, `specs/001-workshop-curriculum-platform/review.md`, was found to carry verbatim copies of source from the **private** `workshop` submodule (54 of 62 normalised 10-token windows matched one private Go file). The gate had been green over that file on every run, because it never read it. A human-directed agent found it; no instrument in this repository did.

The exposure window is one command wide, and it is a command this project uses by default: the `commit` wrapper runs `git add .`, which stages **every** untracked file. Write it, run the wrapper, push — and it is in a public repository's history, which is not editable after a push. `scripts/continuation-check.sh` watches neither `specs/**` nor `docs/*.md`, so nothing else raised it either.

Contract of the new mode

property	behaviour
flag	`--include-untracked` — **opt-in**, off by default
candidate set	`git ls-files --others --exclude-standard` unioned with the tracked set
side	**PUBLIC only.** The private corpus stays tracked-only, so the KEY SPACE is unchanged and the two modes differ in exactly one variable: where a key may be *found*
`.gitignore`	**respected** — that is what `--exclude-standard` is for. Ignored paths are build output and caches that are never pushed
submodules	`--others` does not descend into a nested working tree, so each repository still contributes only its own untracked files, under its own prefix
disclosure	the count of untracked files read is printed on **every** run, in **both** modes, and each path is listed. A

property	behaviour
	default run prints `0 file(s) — NOT SCANNED`
JSON	additive `"untracked": { "included", "side", "files_scanned" }`; every pre-existing field keeps its shape
exit codes	unchanged three-valued 0 / 1 / 2

The default did not move — measured, not asserted

The `HEAD` script and the edited script were run against the same tree six minutes apart:

```
# HEAD copy, extracted with `git show HEAD:scripts/verify-content-boundary.sh`
LEAK — 10589 surviving match(es) (prose 10260, short 272, name 57); 0 row(s) also could not be determined   rc=1
# edited script, default mode
LEAK — 10589 surviving match(es) (prose 10260, short 272, name 57); 0 row(s) also could not be determined   rc=1
```

A full `diff` of the two 21.9k-line reports differs in exactly two places, and neither is a finding:

1. the **three added disclosure lines** (`untracked (public) 0 file(s) — NOT SCANNED …`);
2. `CONTINUATION.md:<n>` → `CONTINUATION.md:<n+34>` on every row that cites it — a **uniform +34** on all 54 such rows, because another agent added net 34 lines to that file between the two runs. `git diff --numstat -- CONTINUATION.md` showed `775 140` at the time, confirming the file was under concurrent edit. Normalising public line numbers away leaves the three disclosure lines as the *only* difference.

What the new mode finds on this tree

```
bash scripts/verify-content-boundary.sh --include-untracked
LEAK — 10633 surviving match(es) (prose 10297, short 279, name 57); 0 row(s) also could not be determined   rc=1
untracked (public)    4 file(s) read
```

+**44 rows** over the default (+37 prose, +7 short, +0 name), from **2** of the 4 untracked files, against **10 distinct private sources** in **2** private submodules:

untracked public file	**rows**	**private sources it matches**
`specs/001-workshop-curriculum-platform/review.md`	22 prose + 2 short = **24**	`workshop/docs/session-evidence/session-ledger.md` (21), `workshop/docs/training/areas/02-…` (`.md` and `.sections.json`, 2), `workshop/platform/orchestration/go.mod` (1)
`docs/session-instruction-audit-2026-09-01.md`	15 prose + 5 short = **20**	`workshop/scripts/setup.sh` (12), `workshop/pipeline/extract/verify.py` (2), `workshop/pipeline/extract/verify_export.py` (2), `workshop/docs/knowledge-model-contract.sections.json` (1), `workshop/docs/session-evidence/search-defect-evidence.md` (1), `ai_interviewing/docs/interview-preparation/areas/22-…` (`.md` and `.pdf`, 2)
`_tests/env.js`	0	—
`specs/001-workshop-curriculum-platform/redaction-review-summary.md`	0	—

Two facts about that table are worth stating plainly rather than leaving to be inferred. First, `workshop/platform/backend/pkg/answer/verify.go` — the private file that `review.md` was paraphrased away from earlier

the same day — appears **nowhere** in it, which is independent corroboration that the paraphrase held. Second, the 24 rows `review.md` still produces are against a **different** private source that nobody had looked for. Fixing one overlap is not evidence about the others; only the scan is.

These numbers are a dated observation on a tree under concurrent edit. Re-run rather than quoting them. The rise is expected and is not to be tuned down: the whole point of the flag is that these rows exist and were invisible.

Paired proof (§1.1)

`--prove-failure` grew from 22 to **24 mutations**, and the summary keeps the `M<n>` / "24 mutations run" form the meta-check's hollow-proof heuristic looks for. Every new case is a **pair against the same tree**, differing only in the flag — without that shape a green result could come from the fixture being clean rather than from the flag doing anything.

case	expected	what it holds open
`M23a`	rc 0	the flag alone must not redden a clean tree. If `--others` ever started descending into the private gitlink, the private corpus would be scanned as public content and every key would match itself
`M23b0`	`??` in `git status`	the seeded file really is untracked, so the case cannot pass for the wrong reason
`M23b`	**rc 0 — the defect**	identical bytes to `M1`, in the same directory; the ONLY difference is that the file is untracked, and the default mode is blind to it

case	expected	what it holds open
`M23c`	rc 1	`--include-untracked` catches it, and **names both sides**, the untracked public path included
`M23d`	rc 0	an ignored path is not read even with the flag
`M23d2`	rc 1	the same bytes in a **non**-ignored untracked file are caught — so `M23d`'s rc 0 is the ignore rule, not a dead detector
`M23e`	identical manifest	the battery **restores** what it seeded, byte for byte (`cksum` over every non-`.git` file, POSIX rather than GNU `sha256sum`)
`M23e2`	rc 0	the restored tree scores clean again
`M24a`	**rc 0 — declared cost**	an UNTRACKED **private** file is still not a key source. A real remaining blind spot, written as a test so a later change that quietly widens the private side goes red
`M24b`	rc 1	commit that same private file and the same public copy is caught at once — so `M24a`'s rc 0 is the tracked-only rule, not a dead detector

Measured **twice**, because `--run-proofs` does not invoke a proof the way a human does — it appends `--quiet` to any script that declares that arm, and that is precisely what turned `submodule-remote-sync` into a hollow-proof false positive:

```
bash scripts/verify-content-boundary.sh --prove-failure
  43 passed, 0 failed, 24 mutations run   rc=0   368 s
bash scripts/verify-content-boundary.sh --prove-failure --quiet
  43 passed, 0 failed, 24 mutations run   rc=0   233 s
```

Both outputs match the meta-check's needle (`(^|[^A-Za-z])M[0-9]|[0-9]+[[:space:]]+(mutation|mutations|…)`) **44 times**, so this check does not become a fifth false positive. It cannot: `run_prove` emits through `printf`/`echo` and uses the `QUIET`-gated `say`/`vsay` helpers **zero** times, so `--quiet` reaches only the nested scans whose output is captured into a variable, never the battery's own report.

The proof's own live-tree pre-flight is REPORTED, never gating, and it recorded `rc=1 (LEAK — 10564 …)` on this tree — a statement about the tree, not about the battery. The count differs from the 10589 measured 20 minutes earlier because the tree is under concurrent edit, which is exactly why these figures are dated observations rather than facts.

Every fixture string is synthetic and was written for the proof. No heading, sentence or name from any real private document appears in it — a fixture carrying the real leaked content would re-leak it into this public repository, and the proof would become the incident.

`name-in-path` — added 2026-09-02

field	value
row	`check name-in-path scripts/verify-name-in-path.sh flag --prove-failure --root /nonexistent`
entry point	`bash scripts/verify-name-in-path.sh`
paired proof	`bash scripts/verify-name-in-path.sh --prove-failure` — **14 mutations**
rc-2 probe	`--root /nonexistent`
exit contract	

field	value
	0 clean · 1 a name-shaped leaf under a role container · 2 could not determine

What it closes

A personal given name belonging to a third party was found on 2026-09-02 in a **tracked, committed, pushed** file of this **public** repository. It was not in a sentence. It was **the final component of an absolute filesystem path**, in a table of indexed directories, under a work-assignments parent:

```
.../Projects/<assignment-ish directory>/<a person's given name>
```

Two instruments were in a position to see it and neither did, for two different reasons — and the pair is the point, because “we have gates” was true the whole time:

instrument	why it missed	fixed?
`audit-hardcoded-paths.sh`	**scope, not detection.** Both disclosed lines MATCH its own `PATTERN`. Its `SKIP` regex began `^(docs/` and is applied **before** the pattern, so of 39 tracked `docs/**.md` files, **zero** were read.	yes — `docs/` removed from `SKIP` the same day; see the allow-list’s own `docs/` block for the measured before/after
`verify-content-boundary.sh`	**question, not scope.** It hunts COPIED PRIVATE CONTENT. A path is not copied content and a name inside a path is not prose.	not applicable — a different question

So `name-in-path` asks a **third** question that nothing in this tree asked before: *does a filesystem path in this repository name a human?*

The signal, and why both halves are required

A hit needs BOTH:

1. the **parent** component is a **role container** — a directory whose name says its children are people or work assigned to people; and
2. the **final** component is **name-shaped** — one bare ASCII alphabetic token, 3–20 characters, no digits and no separators, `Capitalised` or `lowercase`.

plus an **anchoring** rule: the run must be a genuine absolute path (the `/` that opens it is a filesystem root, not part of a URL, a variable, a ratio or a relative path) with at least two components.

No personal name is written in the script, in any fixture, or in any test, and none may ever be. A detector shipping a roster of real names is itself the disclosure it was built to prevent. Both vocabularies are role nouns and generic technical words; print them and check rather than believing it:

```
bash scripts/verify-name-in-path.sh --vocab
```

The precision/recall trade, stated as a measurement

The anchoring rule is not a tidy-up. It was added after measuring the naive form on this tree:

form	findings on this tree	what they were
role-container parent + name-shaped leaf, **no anchoring**	**135**	almost entirely English *alternations* in prose — `client/server`, `person/entity`, `reviewer/author` — and relative asset URLs in CSS. Not paths at all.
the same, **anchored to a real absolute path**	**1**	`/home/<service-account>` in a `Containerfile`, created by `useradd` two lines above it

135 → 1 is the whole design decision, and it is a decision **for precision and against recall**, taken deliberately. The failure mode of a name detector is not a missed name; it is a noisy gate everybody learns to

ignore, after which it catches nothing at all. What that buys is stated as blind spots **B1–B8** in the script header and **printed on every single run**, never left implicit — the largest being B1 (a name whose parent is not a declared role container is invisible) and B8 (a relative path is invisible).

Two subtractions, each with its cost written down:

- **S1** — a closed vocabulary of generic technical words in the leaf position, plus every container word and its `+s` plural (a container name is never a person). *Cost:* a person whose name collides with a generic word is missed.
- **S2** — this repository's **own** public identity, **derived at run time and never written down**: alphabetic tokens from `git config user.name` / `user.email`, every remote URL (root and submodules), the `.gitmodules` URLs, and the components of this checkout's own absolute path. The owner's own account name under `home/` is not a third-party disclosure, and hard-coding it would put a real name in a tracked file. *Cost:* a third party sharing a token with the owner's identity is missed. Reported as a **count** (32 tokens on this tree, 2026-09-02); the tokens themselves are never printed or stored.

Every surviving hit carries a **corpus frequency** — how often that same token occurs as any component of any anchored path anywhere in the scanned universe. A leaked personal name occurs once or twice; a service account occurs all over the file that created it. It is **reported, never a filter**: as a filter it would let a leak hide behind a busy repository.

The output never reprints the suspected name

A gate that echoes the token it just found into a terminal, a CI log or a pasted report has *widened* the disclosure. Findings print as

```
<file>:<line>  <container>/<REDACTED len=N case=Capitalised|
lowercase>  [leaf token occurs K× as a path component corpus-wide]
```

and the reader opens the file. **All 14 mutations assert this invariant**, not just one: every case additionally fails if the planted token appears anywhere in the detector's output.

Measured on this tree, 2026-09-02

```
bash scripts/verify-name-in-path.sh
     no personal-name-shaped final path component under any role-
container
```

```
    directory (1 explicitly allowed)
  scanned 10011 file(s) across 14 repositories        rc=0
```

The one allowance is `submodules/containers/pkg/emulator/Containerfile:75`, a `WORKDIR /home/<account>` whose account is created by `useradd` at line 73 of the same file and occurs **31×** as a path component corpus-wide.

It is LINE-scoped, and that is the default in `.name-in-path-allow` — a deliberate difference from the sibling allow-lists. This gate exists because one line of one file disclosed a real person; a file-scoped pardon on a file that once carried a name is exactly the mechanism that would hide the next one. The cost is stated rather than discovered: a gitlink bump moves the line, the entry goes stale, the detector says so, and the finding returns as a failure. For a privacy gate a false red costs a minute of reading and a false green costs a permanent public disclosure.

Verified against the real incident shape

Not asserted — measured. The redacted line was reconstructed in a throwaway repository with an **invented** given name substituted for the redaction, and the detector run against it:

```
   docs/…/LUMEN-STORE-INVENTORY.md:1  assignments/<REDACTED len=9
case=Capitalised>
   [leaf token occurs 1× as a path component corpus-
wide]                     rc=1
```

It fires, the frequency reads `1×` — the once-or-twice signature of a leaked name rather than the 31× of a service account — and the output does not contain the planted token. The throwaway repository was deleted; no name, invented or otherwise, was written into any tracked file.

Paired proof

```
bash scripts/verify-name-in-path.sh --prove-failure     # rc 0, 14
mutations
```

Shape copied from `audit-hardcoded-paths.sh --prove-failure`: the CONTROL is a **synthetic** throwaway repository, green by construction, so no state of this tree can redden the control and silently switch the battery off; the live run still happens first and is REPORTED, never

gating. The planted token is **invented** and is assembled from fragments so the joined `<container>/<token>` literal never exists in the script — otherwise the proof would plant a finding in the detector's own source.

mutation	asserts
N1	an invented given name as the leaf under a role container → **rc 1**, naming the file
N2	the same finding, `# REASON:` allow-listed → rc 0, still naming what it suppressed
N3	the same finding, `# BASELINE:` allow-listed → rc 0, printed loudly as debt
N4	`path:line` scope pardons **that** line
N5	`path:line` scope at a **non-offending** line does **not** pardon → rc 1. Without this, N4 would prove nothing: a pardon that fires regardless of the line is a file-scoped pardon wearing a line number
N6	a GENERIC leaf under the same container → rc 0 (**S1** is live)
N7	the same token under a NON-container parent → rc 0 (**B1**, blind by design)
N8	a hyphenated compound leaf → rc 0 (**B4**, blind by design)
N9	leaf equal to a token of the repository's own **derived** identity → rc 0 (**S2** is live — and it is the *same leaf* that fires in N1, so the subtraction is demonstrated rather than asserted)
N10	an allow entry matching nothing is reported as stale, never silent
N11–N14	four could-not-determine states → **rc 2**: absent target, not a git

mutation	asserts
	tree, empty scan universe, uninitialised submodule
all 14	the output never reprints the planted token

`pretooluse-guard` — added 2026-09-09

`scripts/verify-pretooluse-guard.sh`. The §11.4.234(A)(3) dedicated hook-validation stage for the §11.4.109 PreToolUse guard hook. It closes inventory gap **G12**.

What it closes

G12 read: *"no `PreToolUse` guard is wired, although the canonical guard script is present in the submodule."* The canonical script has been sitting in `submodules/constitution/scripts/hooks/` unreferenced — a guard present in the tree and wired to nothing, which is the same shape as the unused `submodules/containers` gitlink recorded elsewhere in this repository: every gate green over a component nothing consumed.

The hook is now wired in **`.claude/settings.json`**, which is **tracked** (`git ls-files --error-unmatch .claude/settings.json` succeeds). That choice is the whole point of assertion V3 below. The alternative — the host-wide `~/.claude*/settings.json` — would guard this developer's machine and nothing else, reproducing exactly the `.git/hooks/` hole this repository already documents: untracked, so a fresh clone is unprotected and nobody is told.

The hook is **referenced, never copied**. §11.4.109(A) forbids a local copy in terms — *"a copy diverges silently"* — so V2 accepts only a command string that names the canonical submodule path and REJECTS a local copy even when that copy is byte-identical today.

Contract

Three-valued. **2 is never a pass.**

rc	meaning
0	wired, referenced, tracked, and demonstrably firing

rc	meaning
1	a real defect: unwired, wired to a copy, inoperative, over-blocking, untracked, or a missing/mutilated preamble document
2	could not determine: root absent, root not a git tree, or `bash`/`git` unavailable

The assertions, and why V4–V6 are the load-bearing ones

id	asserts
V1	the canonical guard exists at the canonical submodule path
V2	`.claude/settings.json` declares a `PreToolUse` hook command naming **that** path — a local copy is rejected
V3	that settings file is **tracked in git**, so a fresh clone inherits the guard
V4	the wired guard is **executed** against five forbidden probes — force-push, force-with-lease, `--no-verify`, privilege escalation, host-power — and must refuse all five with rc 2
V5	the same guard **allows** a benign `git status` and a non-Bash tool call — a guard that blocks everything is switched off within the hour, so over-blocking is a defect too
V6	the `# guardrails:allow` escape marker does **not** downgrade the host-power class (§11.4.109(A)5, §12)
V7	`docs/AGENT_GUARDRAILS.md` carries both mandated headings and the `11.4.109` anchor literal (§11.4.109(B)/(C))

id	asserts
V8	the upstream hermetic harness `test_guard_forbidden_commands.sh` is present

V4 is the reason this check exists at all. A validator that reads a JSON key and reports green has verified that a *string* is in a *file*; it has verified nothing about whether a forbidden command would be stopped. §11.4.201 names that failure precisely — a guard that reports a false GREEN is worse than no guard, because it converts an open hole into a closed ticket. Every probe V4 sends is harmless even if the guard were absent: nothing is executed, the command text only ever reaches the guard on stdin as JSON.

Deferred, not dropped (§11.4.234(C)). V8 asserts the upstream harness is PRESENT; it does not run it, because a 70-case suite does not belong inside a pre-push cheap check. Its execution is a separately named stage and the green output says so on every run rather than letting the deferral go unrecorded:

```
bash submodules/constitution/scripts/hooks/
test_guard_forbidden_commands.sh
```

Measured on this tree, 2026-09-09

```
  PASS  V1 canonical guard present     submodules/constitution/
scripts/hooks/guard-forbidden-commands.sh
  PASS  V2 PreToolUse hook wired       → submodules/constitution/
scripts/hooks/guard-forbidden-commands.sh
  PASS  V3 wiring survives a fresh clone   .claude/settings.json
is tracked
  PASS  V4 guard fires on a forbidden probe   5/5 classes refused
with rc 2
  PASS  V5 guard passes benign calls          benign Bash + non-
Bash tool both rc 0
  PASS  V6 escape hatch cannot unlock host-power   still rc 2 with
the marker present
  PASS  V7 preamble document           both mandated headings +
anchor literal present
  PASS  V8 upstream hermetic harness present
  PASS=8 FAIL=0
UNDET=0                                                    rc=0
```

Paired proof

```
bash scripts/verify-pretooluse-guard.sh --prove-failure   # rc 0, 15 cases
```

The CONTROL is a **synthetic** throwaway tree built in `mktemp -d`, green by construction, so no state of the real repository can redden the control and silently switch the battery off. Every mutation is applied to that throwaway; nothing in `submodules/constitution` is ever written to.

case	asserts
`C1`	an intact synthetic tree passes — rc 0
`M1`	the `hooks` key removed from settings → rc 1
`M2`	wired to a LOCAL COPY of the guard instead of the submodule path → rc 1, even though the copy is byte-identical
`M3`	the guard **neutered** to `exit 0` → rc 1. This is the mutation the whole check is for: settings unchanged, config assertion still green, guard useless
`M4`	the guard **over-blocking** with `exit 2` → rc 1
`M5`	a guard that lets `# guardrails:allow` unlock host-power → rc 1
`M6`	settings.json UNTRACKED → rc 1 — the `.git/hooks/` hole, caught
`M7`	the canonical guard script deleted → rc 1
`M8–M10`	preamble deleted / a mandated heading stripped / the `11.4.109` literal stripped → rc 1 each
`M11`	the upstream hermetic harness deleted → rc 1
`M12`	settings.json malformed, so the harness would load no hooks at all → rc 1

case	asserts
U1, U2	not a git tree, and an absent root → **rc 2**, never a pass

M3 and M6 are the two that a config-reading validator would miss entirely, and they are the two most likely real-world regressions: someone edits the submodule copy, or someone gitignores `.claude/`.

One defect the battery found in its own validator

The first run reported `14 passed / 1 failed` with the CONTROL red and the mutation labels printed blank. Two real bugs, both found by the proof rather than by reading: the synthetic settings heredoc emitted `\$` inside a JSON string, which is an invalid JSON escape, so `jq` refused the file the control depended on; and the V4 probe loop read into an undeclared `label`, clobbering the caller's variable of the same name in every case that reached V4. Both are fixed and the second carries an inline comment saying why the `local` is load-bearing. Recorded because a proof that found nothing on its first run is the one to distrust.

`zero-findings-sweep` — registered 2026-09-05, documented 2026-09-09

field	value
row	`check zero-findings-sweep scripts/audit/zero_findings_sweep.sh flag --prove-failure --root /nonexistent`
entry point	`bash scripts/audit/zero_findings_sweep.sh`
paired proof	`bash scripts/audit/zero_findings_sweep.sh --prove-failure`
rc-2 probe	`--root /nonexistent` — **measured rc 2**, 2026-09-09
anchor	§11.4.261 (+ clause (C) ratchet)

This section is the R5 documentation half, written **2026-09-09**, four days after the row itself landed. The row has been conformant since it was added — `verify-check-registry.sh` reports it PASS — but the registry doc carried no entry for it, and a check nobody can read about is a check nobody can audit.

What it enforces

It iterates the §11.4.261(A) closed vocabulary — shortcomings, gaps, weak-spots, danger-zones, todo-fixme, skipped-tests, bluffs, unresolved, divergent-stale-orphan, uncatalogued — over every tracked file at this root, and compares each class's live count against the ceiling recorded in `docs/findings/zero_findings_ratchet.tsv`. Scope is by construction, not by filter: it enumerates with `git ls-files`, and a gitlink is one entry, so submodule content never enters the count. Three exclusions are declared and printed on every run, one of which is the sweep's own source — it carries the patterns it hunts for, so scanning itself would be self-detection, not a finding.

Contract

Three-valued, and a 2 is never a pass:

- **0** — every class is within its ceiling.
- **1** — at least one class ROSE above its ceiling. The seam is refused.
- **2** — COULD NOT DETERMINE: no ratchet snapshot recorded, a detector input missing, or the root absent. An unrecorded baseline is never a pass, and a blind detector reports blind rather than zero.

`--write-ledger` regenerates `docs/findings/zero_findings_ledger.jsonl` (the §11.4.261(B) SSoT). `--write-ratchet` regenerates the ceilings and **refuses to raise any of them**, which is clause (C) implemented rather than described.

Paired proof

`--prove-failure` — **15/15 mutations caught, measured 2026-09-09**, every one DATA (planted fixtures in a throwaway tree), never an edit to the sweep, so no mutation can leave a weakened detector behind. It carries a CONTROL that proves the proof is not vacuous, one mutation per vocabulary class (M1–M10), and three that matter more than the rest:

case	asserts
M11	a finding above the ceiling makes the sweep REFUSE (rc 1) — the rise fails closed
M12	`--write-ratchet` REFUSED to raise a ceiling (rc 1) — debt cannot be absorbed by re-baselining upward
M13	a tree with no ratchet snapshot returns rc **2**, not rc 0
M14	removing a detector input returns rc **2** — the DROP direction, so a detector that stops detecting is reported blind rather than as zero findings

M14 is the case that keeps this sweep out of the trap the constitution sweep itself fell into before 2026-09-02 (see *The ledger, and why it is not a hardcoded list* above): a count that falls because the instrument stopped looking must never read as progress.

`--selftest` is separate and cheaper: **22 passed / 0 failed**, golden-good clean, golden-bad detected in all 10 classes, plus negative controls.

Measured on this tree, 2026-09-09 — and it is RED

exit 1. 2860 tracked files scanned after 2 declared exclusions:

```
shortcomings 1 > 0 | skipped-tests 16 > 9 | TOTAL 35 > 26   ->
RATCHET REFUSED
```

The other seven classes are within ceiling. This red is the ratchet working, not the sweep failing, and it is **left red deliberately**: the ceilings may only ever be lowered, and neither risen class can be honestly cleared from here. The one `shortcomings` row is the registry's own remaining DEBT row (`translation-review`, which owes a proof and a three-valued exit and needs a reviewer model backend). The `skipped-tests` rise is mostly `_tests/preflight.js` and `_tests/prove-preflight.sh`, added 2026-09-06 — real new matches in real new files, not a detector change.

The stale-SSoT defect this section records

Until 2026-09-09 the committed ledger was the one written **2026-09-04** and recorded 26 rows while the live sweep measured 35. Because `CM-ZERO-FINDINGS-MONOTONE-RATCHET` reads the *ledger*, and the README zero-findings badge reads the *ledger*, both were **green/amber over a stale SSoT** while the live sweep refused the seam. Regenerating the ledger flipped the gate to its honest FAIL and the badge to its honest colour. Nothing was weakened; a false green was removed. **A ledger that is not regenerated is not a record, it is a memory of a tree that no longer exists.**

`badge-computer` — registered 2026-09-05, documented 2026-09-09

field	value
row	`check badge-computer scripts/badges/compute_badges.sh flag --prove-failure --check --root /nonexistent`
entry point	`bash scripts/badges/compute_badges.sh --check`
paired proof	`bash scripts/badges/compute_badges.sh --prove-failure`
rc-2 probe	`--check --root /nonexistent` — **measured rc 2**, 2026-09-09
anchor	§11.4.259 (clauses A, C, D, E, F)

What it enforces

§11.4.259(C) is the clause that decides what this script is: every badge's colour and value is COMPUTED from a live source of truth, never hand-typed. The script separates MEASURE (run commands against this tree), CLASSIFY (turn a measurement into one of the closed GREEN/AMBER/RED vocabulary) and RENDER (emit Markdown), so the (F) golden fixtures can exercise classify and render with planted inputs without first breaking the real repository. `--check` re-runs the

measurements and REFUSES when the committed row no longer matches them; `--write` regenerates the row in `README.md` and the provenance table in `docs/BADGES.md`.

Contract

- **0** — the committed row matches every live source.
- **1** — DRIFT: the row disagrees with a measurement, is missing, or a badge has no provenance entry.
- **2** — COULD NOT DETERMINE (e.g. the README is absent). Never a pass.

Paired proof

`--prove-failure` — **9/9 mutations caught, measured 2026-09-09**, including the CONTROL. The load-bearing ones are M1 (a 0% measurement against a 90% target classifies RED, not GREEN — the §11.4.259(F) golden-RED assertion, the bluff the anchor names explicitly), M2 (the colour `beta` is REFUSED — clause (A) admits no gradations), M4 (a stale row claiming green over a red measurement → rc 1), M5 (an absent README → rc 2, not 0), M6 (the production-readiness gauge over `{amber,red}` inputs is RED — it cannot outrank its inputs) and M8 (all 12 badges carry a non-empty provenance string).

`--selftest` runs the clause-(F) fixtures: **12 passed / 0 failed** — golden-good GREEN, golden-bad RED, golden-amber AMBER, plus negative controls asserting the colour actually reaches the rendered URL.

Measured on this tree, 2026-09-09 — the row was STALE and understated the red

`--check` opened at **rc 1**. The committed row claimed `evidence 22/22 proofs` (amber) and a gauge of `blocked, 8 red`; the live measurement said `evidence 29/30 proofs` (**red**) and `blocked, 9 red`. The README was therefore showing a **friendlier** picture than the tree — the §11.4.259(E) always-in-sync violation, in the direction that matters. `--write` regenerated it and `--check` now exits **0** (12 badges in sync).

The row is honestly red — 9 RED, 2 AMBER, 1 GREEN, gauge RED — and that is compliance, not failure. §11.4.259's own honest boundary says so: *"a project whose badge row is HONESTLY red is compliant with §11.4.259 (the reader gets the truth) even while it fails other release*

gates." The alternative — omitting the classes this project has no instrument for — is the §11.4.201(6) FALSE-NULL the anchor names by name.

One defect this session's re-measurement found in the badge computer itself

Regenerating the row surfaced a fault the earlier green `--check` could not have shown, because it only appears once a badge CHANGES colour. Both the `zero-findings` and `evidence` badges built their provenance string with the rationale **hardcoded**: `"... AMBER because the §11.4.261 invariant is ZERO and the ratchet is holding at a brownfield baseline"` and `"... AMBER not GREEN because §11.4.262 also requires a CAPTURED evidence ARTIFACT"`. The colour was computed by a three-branch conditional directly above; the sentence explaining it was not. So when the ledger rose past the ratchet and the badge correctly turned **RED**, `docs/BADGES.md` went on telling the reader why it was AMBER.

§11.4.259(C) makes provenance part of the badge, not a comment beside it — a badge whose provenance is wrong is not a badge with a cosmetic blemish. Both rationales are now selected in the same branch that selects the colour, so a rationale cannot disagree with the badge it explains. Re-measured after the fix: `--selftest` 12 passed / 0 failed, `--prove-failure` 9/9 caught, `--check` rc 0.

Worth recording for the class, not the instance: **a value and the sentence explaining that value must be produced by the same branch.** Anywhere they are computed separately, they are free to drift, and they will drift silently in the direction of whatever the sentence was written for on the day.

`claim-ledger` — added 2026-09-09 (§11.4.266)

`docs/claim-ledger.tsv` + `scripts/verify-claim-ledger.sh`

Registered as `claim-ledger`, proof `--prove-failure`, rc-2 probe `--root /nonexistent`.

What it closes — nine measured instances, not a hypothesis

No instrument in this tree compared a RECORDED CLAIM against the thing it describes. Every gate here verifies its subject; not one verified what the repository *says* about its subject. A gitlink moves, every gate stays green, and the documents go on asserting a gap that closed days ago. Nine instances were measured on 2026-09-07/08/09:

1. `submodules/containers` — nine tracked files asserted "`pkg/runtime` has no `Run`, no `Create`" for five days after the primitive shipped upstream, two commits *before* the pin this tree already consumed.
2. Three "actionable FAILs" recorded as work to do; all three already existed.
3. Two of those gates were **PASSING over a stale ledger** — it recorded 26 findings while the live sweep measured 35, and both the ratchet gate and the README badge read the LEDGER rather than the live sweep.
4. `-question-verify none` recorded as unrecognised; it had landed in `5642dd6`.
5. A `redactions` table recorded as empty; it had already materialised.
6. "34 runs no setting readable today explains" — it is 43, and they are.
7. "the sweep keeps no expected-gate ledger" — the ledger exists, at 271 rows.
8. A note claiming 66 rows against a 49-row artefact, whose gate exited 0 because nothing read the prose.
9. A stop-rule asserting a PROXY that fires on the safe case.

Note the shape: several were GREEN GATES OVER STALE GROUND TRUTH. That is the failure this gate must not become, and the format is built against it.

Why a side ledger and not in-place annotation

Both were considered. The ledger won on three measurements:

- **The four-carrier lockstep constraint.** `CLAUDE.md` / `AGENTS.md` / `QWEN.md` / `GEMINI.md` are byte-identical from the measured convergence line (19) down and cascade check **C5** enforces it. An in-place annotation would have to be byte-identical in four files, so *registering one claim* would become a four-file lockstep edit — and every such edit is a chance to put C5 red. Registration must be cheap or it will not happen.

- **§11.4.266(A) mandates the ledger shape**: “ONE LEDGER PER REPOSITORY, ENUMERATED FROM THE CLAIM SIDE.” A row is the unit the anchor names.
- **This repository already parses ledgers as TSV** — `scripts/check-registry.tsv`, `scripts/constitution-gate-ledger.tsv`, `docs/findings/zero_findings_ratchet.tsv`. TSV needs no `yq`, and a pre-push instrument must not have a parser that can fail to load.

The cost of the choice is stated rather than hidden: a side ledger can be orphaned from the document it describes. That is exactly why **L1** exists — a row whose anchor no longer matches is a FINDING, not a tidy-up.

The format, and the one property that keeps it honest

```
surface  <path>
claim    <id> <surface> <kind> <bluff-type> <severity> <op>
<expected> <anchor> <probe>
```

A `prose` row stores NO measured figure. The CLAIMED value is read out of the document at run time by the anchor’s single capture group; the REAL value comes from running the probe fresh. There is nothing cached that can go stale, and **no way to make such a row green by editing the ledger** — editing the anchor only re-points it at different prose. Instance 3 above cannot recur in a `prose` row by construction.

An `assert` row is for a claim that states no number (“the sweep keeps no expected-gate ledger”). It *does* store an expected value, and that is its honest boundary: someone could make it green by editing `expected` while the prose still asserts the old proposition. Prefer `prose`.

`bluff-type` is drawn from the **seven-member closed vocabulary of §11.4.266(B)** — `green-but-broken`, `coverage-theater`, `rubber-stamp-verified`, `stubbed-core`, `doc-vs-code-drift`, `config-present-but-unwired`, `byte-identical-fork`. An invented type is a FAIL. The type is a **routing key** to the anchor that owns its counter-gate (§11.4.266(C)), not a label. `severity` is consumer-supplied DATA — the corpus states no scale; here it is `blocker | major | minor`.

Contract

Three-valued, and **2 is never a pass**:

- **0** — every row’s claim is still made and still agrees with a measurement taken during *this run*

- **1** — a claim is FALSE, a row is an ORPHAN, a row carries an invented bluff type, or a row is malformed
- **2** — no `python3`, no ledger, **a ledger with zero claim rows** (vacuity: a gate that reports 0 over nothing has certified nothing), an absent surface, or a probe that would not run

Precedence: 1 outranks 2, and it is asserted by mutation `P1` rather than declared. If 2 won, one broken probe anywhere would mask every false claim in the file.

Every occurrence of an anchor is checked, not the first. A dead figure written in a live figure's clothes is a finding. This repository deliberately records superseded numbers in prose, so an anchor must be specific enough to match only the live assertion — a too-loose anchor surfaces as a FALSE, never as silence. Control arm `C1` proves a superseded figure recorded *outside* the anchor stays green, so the withdraw-by-name convention is not made impossible.

Paired proof (§1.1) — 13 arms, data-driven, in a throwaway lab

Nothing in the battery edits the gate, and nothing runs against the live tree. Two controls, eight mutations, three rc-2 arms:

```
C0  CONTROL — a fully TRUE ledger over a matching tree fires
nothing        rc 0
C1  CONTROL — a SUPERSEDED figure in prose, outside the anchor,
stays green rc 0
M1  the DOCUMENT'S asserted figure is
falsified                              rc 1
M2  REALITY MOVES while the claim stays put — THE measured defect
class     rc 1
M3  ORPHAN — the claim is deleted, the row
survives                         rc 1
M4  a SECOND occurrence of the claim shape carries a DEAD
figure           rc 1
M5  an INVENTED bluff type outside the seven-member closed
set              rc 1
M6  a MALFORMED
row                                                          rc 1
M7  an ASSERT row's reality moves — the thing it denies now
exists         rc 1
U1  a probe that WILL NOT RUN is UNDETERMINED, never
verified               rc 2
U2  an ABSENT SURFACE is
UNDETERMINED                                       rc 2
U3  VACUITY — a ledger with zero claim rows certifies
nothing              rc 2
```

```
P1  PRECEDENCE — a FALSE claim outranks an UNDETERMINED
one                rc 1
```

C0 is the §11.4.201(1) golden-FALSE fixture: a fully-populated ledger whose every row is true must fire *nothing*. **M2 is the load-bearing arm.** All nine measured instances have that shape — reality moved, the claim did not — and a gate that caught only M1's direction would have caught *none* of them.

Every arm declares `expect_change`, and an arm whose body leaves the lab byte-identical FAILS. Adopted from the lesson recorded in `workshop/platform/gates/prove-plan-coverage-proposal.sh`, where three arms silently became no-ops when the figures they named literally moved, and only surfaced because they wanted rc 1 — an arm wanting rc 0 would have passed for ever while testing nothing. The harness was itself proved operative: seeding M2's body with `true` in a throwaway copy produces `FAIL M2 … the arm body left the lab BYTE-IDENTICAL`, and the battery exits 1.

What this gate does NOT assert — read before quoting a green exit

- **COMPLETENESS.** §11.4.266(A) requires *every* advertised capability on every declared surface to resolve to exactly one row. **`CM-CLAIM-REALITY-LEDGER-COMPLETE` IS NOT SATISFIED.** The ledger is a bounded slice of 10 rows and both the gate and the ledger say so on every run. `--completeness` counts unregistered countable-claim shapes and reports them as waves for an operator to approve; it is a NOTE, never a verdict input, because a heuristic claim-shape scanner cannot decide what is a claim.
- **ORACLE STRENGTH.** A row proves the claim agrees with what its probe measured, not that the probe measures the right thing (§11.4.245, §1.1).
- **The other three carriers.** A shared-region claim is registered once against `CLAUDE.md`; C5's byte-identity covers `AGENTS.md` / `QWEN.md` / `GEMINI.md`. If C5 went red, a now-false claim surviving in one carrier alone would be invisible *here*. The two gates are complements.

First real run, 2026-09-09 — RED, on three genuinely stale claims

```
   rows:    10 claim row(s) over 5 declared surface(s)
   verdict: 7 VERIFIED · 3 FALSE · 0 ORPHAN · 0 MALFORMED/UNTYPED
· 0 UNDETERMINED
```

The three FALSE rows were real and were **not** seeded:

row	the document said	a fresh measurement said
`carriers-containers-consumer-files`	`_tools/containers/` has **7** tracked files	**18**
`carriers-sweep-keeps-no-gate-ledger`	the sweep keeps **no expected-gate ledger**	it exists, 271 rows
`readme-zero-findings-badge`	badge: `zero findings — 35 over ratchet`	live sweep TOTAL **33**, then **34**

The last row is instance 3 recurring while this gate was being written: the badge was computed once and frozen, the sweep kept moving, and the README went on showing a friendlier picture than the tree. It also moved **between two runs of this gate in the same session**, which is why the gate fingerprints its read set and reports a moving corpus by path.